

CAWSR: Carla-AutoWare Scenario Runner

David Gasinski ¹, Olek Osikowicz ¹, Gwilym Rutherford ¹, and
Donghwan Shin ¹

¹ The University of Sheffield

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

CAWSR (CARLA-AutoWare-Scenario Runner) facilitates the simulation-based testing of the open-source autonomous driving system, Autoware, within CARLA, the state-of-the-art open-source driving simulator. Building on existing tools, this project introduces a research-oriented testing framework for the execution of complex driving scenarios, as well as supporting implementation of a wide range of verification strategies.

Statement of Need

Verifying Autonomous Driving Systems (ADS) is a critical step before they can be deployed. However, relying only on real-world testing is too expensive, inefficient, and potentially dangerous. Consequently, simulation-based testing has become essential, allowing researchers to safely test driving agents against critical situations at scale. Among these tools, CARLA ([Dosovitskiy et al., 2017](#)) has become the de-facto standard in the research community due to its rich ecosystem of open-source tools, benchmarks, and documentation.

Currently, the standard for evaluating ADS in CARLA is the CARLA Leaderboard and its engine, Scenario Runner (SR) ([CARLA, 2025](#)). This framework is typically used to test “black-box” driving agents, such as those based on Vision Language Models or Reinforcement Learning (DS: add refs here). By running a set of predefined, challenging driving scenarios, researchers can systematically assess agent performance using common metrics like driving score, infractions, and route completion. However, applying this testing framework to industry-grade ADS, such as Autoware ([Kato et al., 2018](#)) or Apollo ([Baidu, 2017](#)), remains difficult. Although communication bridges exist between CARLA and these systems ([Guardstrikelab, 2023](#); [Kaljavesi et al., 2024](#)), they lack native support for scenario execution engines, which limits their utility for scenario-based testing.

This gap has created a significant bottleneck for the research community. Previously, researchers developing scenario generation algorithms mainly relied on combining Apollo with the LGSVL simulator ([Rong et al., 2020](#)). However, LGSVL is now outdated, with official support ending in January 2022. This leaves many researchers without a suitable industry-grade “subject” for evaluating their algorithms. While recent tools like PCLA ([Tehrani et al., 2025](#)) attempt to simplify deploying Autoware (and other ADS implementations) into CARLA, they focus primarily on simplifying the ADS implementations and abstracting the setup process across different CARLA versions. They lack the deep integration required between the agent and simulator to execute complex, route-based scenarios.

CAWSR aims to bridge this gap by enabling the evaluation of Autoware in complex driving scenarios within CARLA. By building on the established CARLA platform, this work provides a modern replacement for the outdated Apollo/LGSVL workflow. It also allows Autoware to be directly compared with state-of-the-art research agents on the CARLA Leaderboard.

Effective ADS verification requires the ability to systematically explore the operational design domain. To support this, CAWSR provides a flexible interface for algorithmic scenario generation. This facilitates a wide range of verification strategies based on common metrics, such as the CARLA Leaderboard's driving score (CARLA Team, 2024).

Lastly, it is worth noting that simulators can often introduce unintended nondeterminism, which leads to inconsistent test results (Chance et al., 2022; Osikowicz et al., 2025). Therefore, CAWSR is designed to minimise such nondeterminism throughout the evaluation pipeline.

Tool Overview

CAWSR is a fully synchronous testing framework that directly integrates the CARLA simulator, Scenario Runner (as the scenario executor), and Autoware (as the System Under Test) to facilitate autonomous driving testing research. The tool is distributed as a Docker container designed and currently supports two modes of operation:

1. *Scenario Generation Mode*: Enables the dynamic generation and execution of scenarios (e.g. iterative scenario generation) provided by a user-defined algorithm. This is particularly useful for assessing the performance of new simulation-based ADS testing techniques.
2. *Benchmark Mode*: Allows the execution of a predefined set of scenario definitions provided by the user. This is useful for standardised evaluations and comparisons between different driving agents.

The evaluation pipeline is engineered to be fully synchronous, minimising unintentional non-determinism to facilitate reproducible results. However, it is noted that minor variations may still persist due to inherent non-determinism in upstream dependencies, such as the driving simulator or the driving agent itself (Chance et al., 2022; Osikowicz et al., 2025).

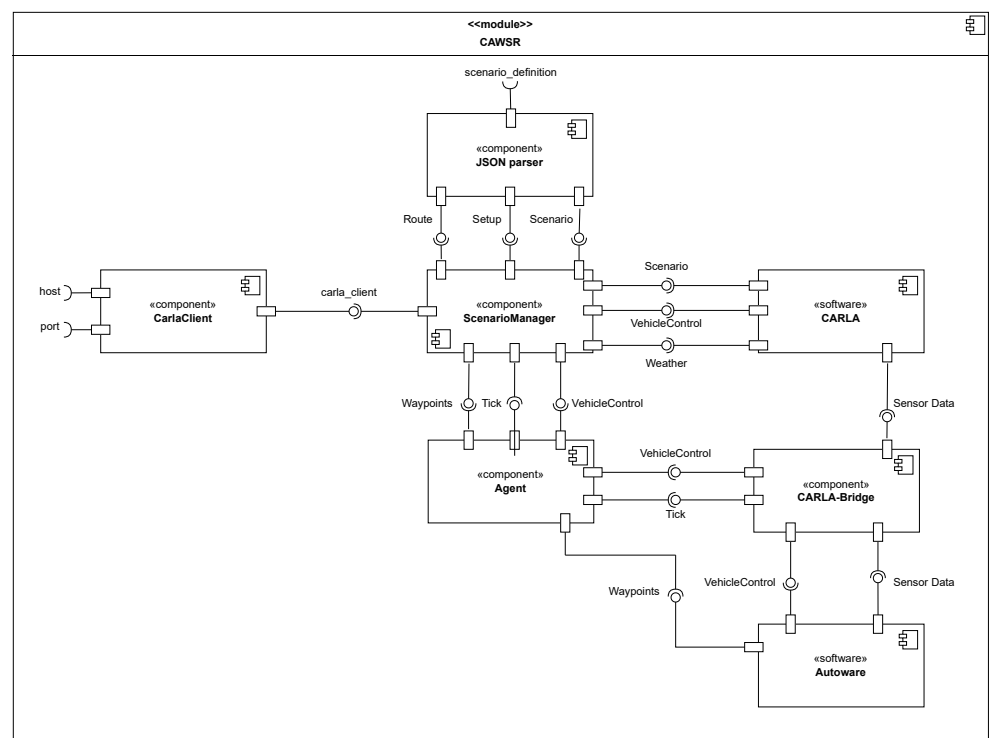


Figure 1: Internal component diagram of CAWSR.

64 Figure 1 shows architecture of CAWSR with the fundamental components of the framework.
65 **CarlaClient**, a native CARLA PythonAPI class, establishes a TCP connection to the simulator
66 via a host *IP* and *port*. As the framework's sole communication link with CARLA, it allows
67 CAWSR modules to extract data and spawn entities by interacting with the internal server.

68 **JSON parser** translates *scenario_definition* into a Behavior Tree (BT) by extracting the route
69 and its associated trigger events. These trees are constructed using Scenario Runner's Atomic
70 Behaviours and Atomic Conditions. Serving as the framework's building blocks, these
71 elements represent discrete CARLA actions and variables—such as spawning a pedestrian—to
72 define the scenario's logic.

73 **ScenarioManager** manages the setup and execution loop, using **CarlaClient** to spawn entities.
74 During each loop, it executes the behavior tree to update actor states and evaluate conditions.
75 It then triggers a simulation tick, advancing CARLA's internal clock and generating a snapshot.
76 This snapshot is passed to **Agent**, which monitors Autoware's internal state and route data.
77 Upon initialisation, Agent connects to Autoware via ROS2. Subsequently, at each step,
78 **CarlaBridge** (Kaljavesi et al., 2024) extracts snapshot data, transforms sensor inputs into
79 Autoware's coordinate system, and publishes them. Finally, Autoware processes this data and
80 issues control commands, which are applied to the ego vehicle.

81 The internal loop within ScenarioManager continues executing until one of the following
82 termination conditions is met, as defined by the CARLA Leaderboard evaluation criteria
83 (CARLA Team, 2024), shown in Table 1.

Table 1: Termination Criteria of each scenario within CAWSR.

Termination Criteria	Description
Route_Completion	Agent reached the end of the route.
Actor_Blocked	Agent is blocked, not moving for 180s.
Simulation_Timeout	No client-server communication established (30s).

84 The same set of standard evalutaion critera is employed to calculate the driving score (DS) for
85 each scenario execution, shown in Table 2.

Table 2: Evaluation Criteria of each scenario within CAWSR, per the CARLA Leaderboard (CARLA Team, 2024). Each criteria applies a fixed penalty to the DS.

Evaluation Criteria
Collisions_with_pedestrians
Collisions_with_other_vehicles
Collisions_with_static_elements
Running_a_red_light
Failure_to_yield_to_emergency_vehicle
Running_a_stop_sign

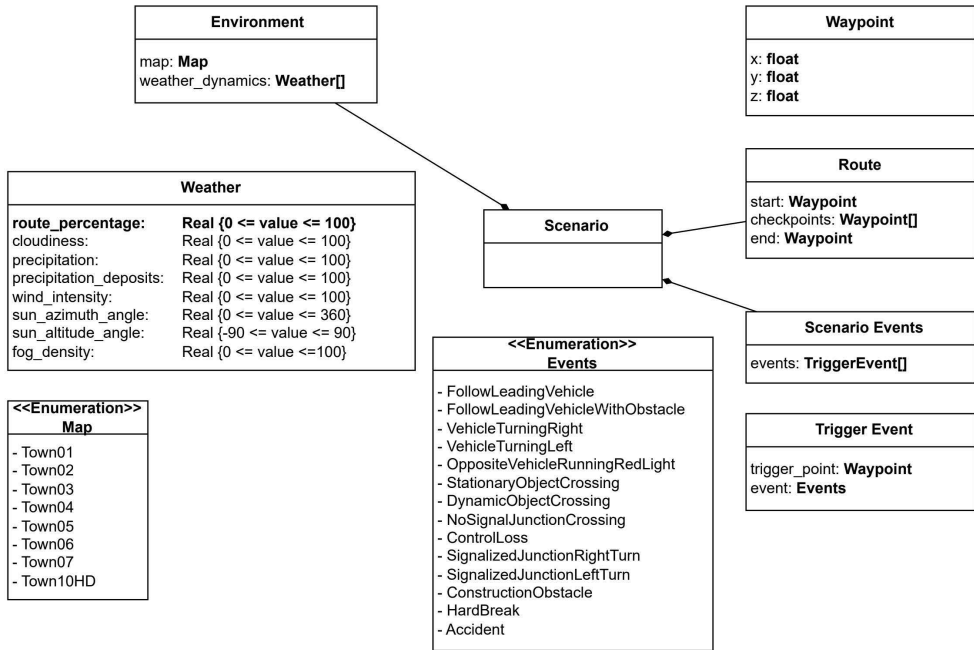


Figure 2: Scenario definition domain model.

86 To facilitate development, we introduce a new domain model for the definition of route-based
87 scenarios within CARLA, described in Figure 4, alongside a JSON implementation. This model
88 is based on the format introduced by Scenario Runner, facilitating support between both
89 frameworks.

90 Conclusion

91 To summarise, CAWSR provides ADS testing research community an easy to use Autoware
92 evaluation pipeline. We hope that this work can facilitate the evaluation of new testing
93 approaches on a state of the art driving system.

94 Acknowledgements

95 This work was supported by the Institute of Information & Communications Technology
96 Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-
97 02218761, 50%) and by the Engineering and Physical Sciences Research Council (EPSRC)
98 [EP/Y014219/1].

99 References

100 Baidu. (2017). *Apollo: Open Source Autonomous Driving*. <https://github.com/ApolloAuto/apollo>.

101

102 CARLA. (2025). *Scenario Runner: Traffic Scenario Definition and Execution Engine for CARLA*. GitHub; https://github.com/carla-simulator/scenario_runner. https://github.com/carla-simulator/scenario_runner

103

104

105 CARLA Team. (2024). *Get started with leaderboard 2.0. CARLA autonomous driving leaderboard*. https://leaderboard.carla.org/get_started_v2_0/

106

- 107 Chance, G., Ghobrial, A., McAreavey, K., Lemaignan, S., Pipe, T., & Eder, K. (2022). On
108 determinism of game engines used for simulation-based autonomous vehicle verification.
109 *IEEE Transactions on Intelligent Transportation Systems*, 23(11), 20538–20552. <https://doi.org/10.1109/TITS.2022.3177887>
110
- 111 Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). CARLA: An open
112 urban driving simulator. *Proceedings of the 1st Annual Conference on Robot Learning*,
113 1–16.
- 114 Guardstrikelab. (2023). *carla_apollo_bridge: Data and Control Bridge for Apollo and Carla*.
115 GitHub; https://github.com/guardstrikelab/carla_apollo_bridge.
- 116 Kaljavesi, G., Kerbl, T., Betz, T., Mitkovskii, K., & Diermeyer, F. (2024). *CARLA-autoware-
117 bridge: Facilitating autonomous driving research with a unified framework for simulation
118 and module development*. 224–229. <https://doi.org/10.1109/iv55156.2024.10588623>
- 119 Kato, S., Tokunaga, S., Maruyama, Y., Maeda, S., Hirabayashi, M., Kitsukawa, Y., Monrroy, A.,
120 Ando, T., Fujii, Y., & Azumi, T. (2018). Autoware on board: Enabling autonomous vehicles
121 with embedded systems. *2018 ACM/IEEE 9th International Conference on Cyber-Physical
122 Systems (ICCPS)*, 287–296.
- 123 Osikowicz, O., McMinn, P., & Shin, D. (2025). Empirically evaluating flaky tests for au-
124 tonomous driving systems in simulated environments. *2025 IEEE/ACM International Flaky
125 Tests Workshop (FTW)*, 13–20.
- 126 Rong, G., Shin, B. H., Tabatabaee, H., Lu, Q., Lemke, S., Možeiko, M., Boise, E., Uhm,
127 G., Gerow, M., Mehta, S., Agafonov, E., Kim, T. H., Sterner, E., Ushiroda, K., Reyes,
128 M., Zelenkovsky, D., & Kim, S. (2020). LGSVL simulator: A high fidelity simulator for
129 autonomous driving. *2020 IEEE 23rd International Conference on Intelligent Transportation
130 Systems (ITSC)*, 1–6. <https://doi.org/10.1109/ITSC45102.2020.9294422>
- 131 Tehrani, M. J., Kim, J., & Tonella, P. (2025). PCLA: A framework for testing autonomous
132 agents in the CARLA simulator. *Proceedings of the 33rd ACM International Conference
133 on the Foundations of Software Engineering*, 1040–1044.